

What is this?

This is a fully hyperlinked and bookmarked PDF excerpt from the upcoming Elevator 2.0 UML Case Study by Leon Starr, i.e. me. Go ahead, click on stuff. This is just a working title and will probably change prior to publication.

Published (for now) by Model Integration, LLC.
www.modelint.com
leon_starr@modelint.com

Dear Readers —

I am making this excerpt freely available to UML developers from the Model Integration website. Feel free to download it and print a copy or two for your own private use. Publication and distribution in hardcopy or electronic form without the author's permission is strictly forbidden. But I am interested in maximizing exposure, so, if you have any ideas, please get in touch with me. I am especially happy to help any academic/university projects.

Also, if you find any errors, or have any technical or other questions, feel free to e-mail me or post on our forum at the website above.

The Elevator 2.0 models in PDF and Nucleus Bridge-Point backup and an executable generated from these models that runs on Windows (easily ported to Linux if there's interest) are all available from the website.

Is any of this useful? Can it be improved? Do you want something completely different? Let me know what you think!

A handwritten signature in black ink, appearing to be 'L. Starr' or similar, with a stylized, flowing script.

— Leon

Part 5

TRACING MODEL THREADS

The fastest way to understand a model is to carefully trace through one or two key threads. Here we will do just that. Complete this exercise and you should quickly get a feel for how the class, state and action models interlock to define a complete executable system. I recommend printing out or displaying all the models to readily examine each component as it becomes involved in the trace. But first, let's clarify thread concepts in the context of Executable UML models.

Model level threads

The user pushes a button in an elevator Cabin. Now what happens? More to the point, what can happen? Intuitively, there is a clear sequence of activity triggered by the button push. But you must also consider allowable parallel sequences and variable activity execution durations because these can vary from one implementation to the next. If a test scenario runs okay on your model test engine this doesn't necessarily mean it will work in the code generated implementation. To avoid synchronization surprises on the target platform you must somehow reconcile all time varying facets of the response behavior with the Executable UML synchronization rules [Mellor 1] sufficiently to *know* that everything is going to work out okay on any given platform.

In addition to the UML collaboration and sequence diagrams I would like to offer my own, which I call a *thread chart* and a *thread overview*. Together, these diagram types can help you make sense of concurrent interactions. Since the usage of standard diagram types in Executable UML are already well documented [Mellor 1], [Carter], I will introduce only the thread chart and overview diagrams here.

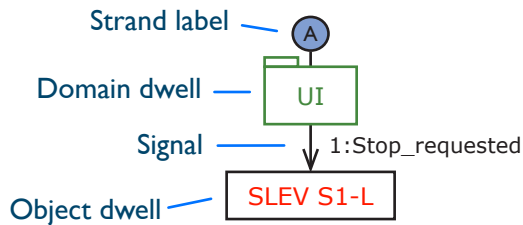
Definition

A model level *thread* is initiated when our domain receives an unsolicited stimulus from an external entity. (This is not the same thing as an *implementation thread*. Our focus lies entirely within the realm of UML models). A thread is the path of behavior through all model elements composing the total response to the externally triggered stimulus. Since objects may execute concurrently, a thread may unwind into multiple *strands* of behavior that can be processed independently of one another. Each strand can further unwind into more strands. Multiple strands may weave back together at point of synchronization. When all strands have terminated, the response is complete. And, yes, all strands must terminate. Otherwise you would have a non-ending response kind of like what my PC operating system does to me whenever it gets the chance.

Now lets move on to some examples of these concepts with notation.

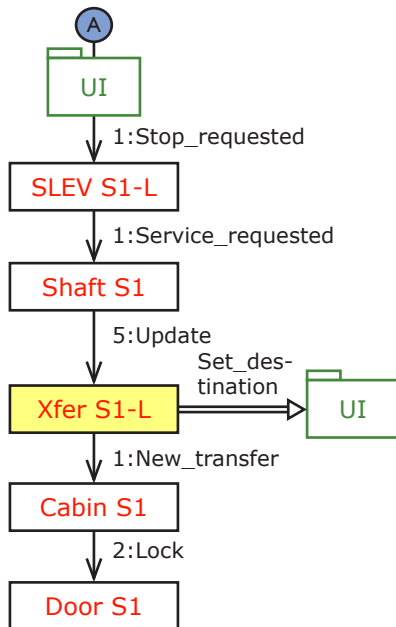
Thread initiation

A thread begins with an unsolicited stimulus triggered by an external entity crosses into our domain.



A button press has been processed by the UI domain and a bridge to the Elevator Application has been invoked. This bridge has generated an SLEV1: Stop_requested signal to the desired destination Shaft Level, instance S1-L (Shaft 1 - Lobby). Since our perspective does not extend into the UI domain, we don't show the bridge call that fired the 1:Stop_requested input signal.

Following the strand further, we see a few more symbols.



Strand A proceeds sequentially triggering activity in more instances (Shaft S1, Transfer S1-L, Cabin S1, Door S1 and so on). Signal parameters have been omitted, but in some cases you might want to include them.

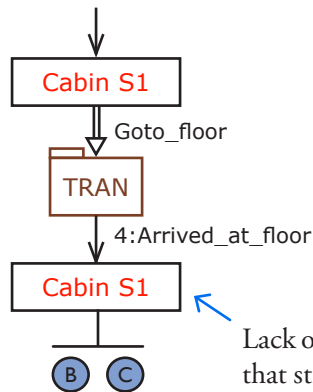
Each object rectangle represents a span of one or more internal state changes until an inbound signal must be received. This introspective phase is called an *object dwell*. Self directed signals are internal to each object and do not appear on the thread chart.

The Xfer S1-L object dwell is shaded to show that the object is newly created. It emits both a bridge call to the UI and a signal to Cabin S1. Since the bridge call terminates immediately in the UI domain, there is no need to spin off a separate strand. Strand A proceeds on through Cabin S1, Door S1 and so on.

In this manner, a strand represents a primarily sequential flow of control. If control branches off significantly, a new strand will be created.

Unwinding and termination

Fast forwarding ahead a bit in strand A, we come to two interesting points. First, we see that the strand extends out of the Elevator Application into the Transport domain and then back with a signal to Cabin. We still have a nice sequential thread because after returning from the Goto_floor bridge call, Cabin S1 waits for a signal to be received. Second, strand A unwinds into strands B and C. This is because Cabin S1 has issued two signals to separate objects.



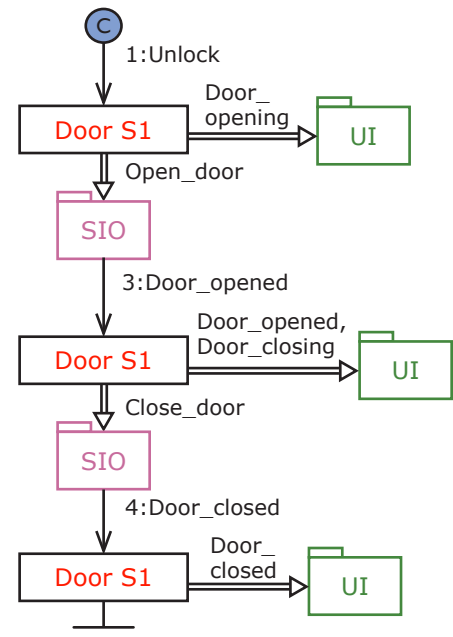
When you test the models, you can choose to defer the processing of all signals destined for the C strand while you advance through the B strand, or vice versa. You can even interleave the processing so that you do a bit of B and then some of C. Be sure to test against a reasonable cross sample.

For now we will just look at the C strand. You can see that a Door1: Unlock signal has been generated by Cabin S1 to Door S1. Then strand C proceeds sequentially until a point of termination is reached as indicated by the parallel line symbol at the bottom. This means that when Door S1 completes its activity and any number of internal state changes, no further incoming signals are processed in this strand.

When each strand has terminated, either inside or outside our domain, the thread is complete.

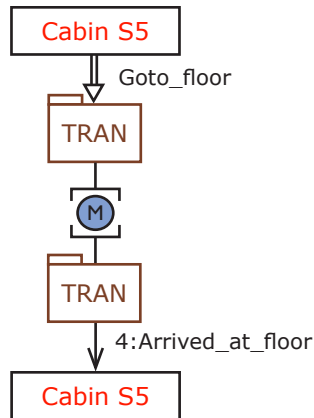
Keep in mind that the thread summary does not represent all possibilities for each object. The state machines and class collaboration show the complete set of possible aggregate interactive behaviors. A thread summary merely illustrates the consequence of a single unsolicited external stimulus under defined test conditions.

So termination doesn't mean that an object is deleted or quiescent. It just means that participation in the test thread has completed.



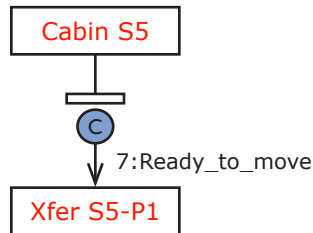
Weaves and knots

You may want to specify a test where a strand initiates and completes during an object or domain dwell period. For example, thread F1 specifies that a stop request is registered while a Cabin is in transit (during the Transport domain dwell period).



Here the M strand must process a stop request after Transport is invoked by the Goto_floor bridge call, but before Transport responds with the 4: Arrived_at_floor signal. To ensure this behavior, the Transport activity must somehow pause while strand M executes and then resumed after M completes. Implementation of the pause depends on the available features in your test environment. In a worst case scenario you could add some temporary instrumentation in your action language. (If you need to do this, you need to get yourself a new test environment!)

A strand may proceed up to a point where it must wait until one or more concurrent strands have completed. This is shown with the knot symbol.



The E strand shown on the left (from thread F1) must wait after completing the Cabin S5 dwell period until the C strand terminates. Upon termination, the C strand will send the 7:Ready_to_move signal to Xfer S5-P1 which causes the E strand to resume. This same situation might also have been handled by having the E strand terminate and having the C strand initiate a new strand, but this would make the logical continuity of the E strand less obvious.

Before moving on to the thread overview diagram, let's take a complete inventory of all thread chart symbols in the following glossary.

Thread chart notation glossary

In proposing the notation I have tried to borrow from UML 2.0 [OMG] where possible. Most importantly, all symbols are whiteboard/pencil friendly. (With the possible exception of the UML package symbol). Comments as to how the notation can be improved or comply more closely with UML are very welcome.

Strand label 

The beginning of each strand is labeled with a short name, typically one or two characters. If the connecting line has no arrow, this strand is the beginning of the thread.

Domain dwell 

A domain dwell represents a duration where activity is taking place in the indicated domain.

Object dwell 

An object dwell represents the duration where operations, state activity or some combination is executed by an object. Multiple state transitions and operation calls may occur inside a single object dwell if no external signals or calls are triggered. Once a signal is issued to a non-self destination, the strand must flow out into some other dwell. For the sake of brevity, minor operations invoked on external objects and calls to domain functions may be omitted and considered internal to an object dwell. The class name or key letter and complete identifier value must appear in the label of the object dwell.

Signal  1:Stop_requested

An Executable UML signal triggered during an object dwell or from a bridge during a domain dwell is shown using a standard UML asynchronous message arrow. The signal label follows Executable UML convention with the omission of the key letter since this is always present in the destination object dwell.

Operation / Domain function call  Estimate_travel_delay

A solid arrow head indicates an operation invocation if it points to an object dwell. The invocation of a local domain function is indicated by pointing the arrow from an object dwell to the dwell of *its own* domain. The solid arrow is the standard UML symbol to indicate a synchronous function call.

Thread chart notation glossary (continued)

Bridge call  Goto_floor

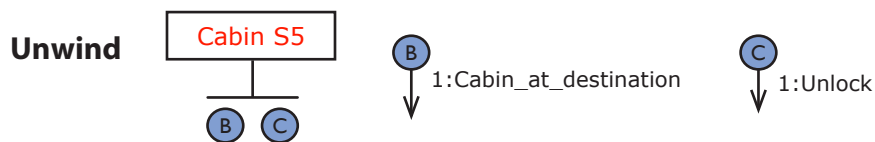
A call to an external domain is shown as a double lined arrow. It must point to a domain dwell.

Object creation and deletion

Xfer S5-P1

Xfer S5-P1

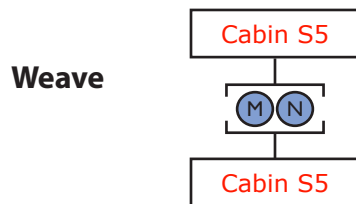
The first appearance of an object created during the thread is indicated with a shaded object dwell. The last appearance, before thread-time deletion is shown with a dashed border object dwell.



A strand may terminate and trigger multiple potentially concurrent strands. The terminating dwell in the source strand appears on the point of the T symbol while the new strands are labeled along the flat end. In the example above, the strand leading up to the Cabin S5 object dwell terminates by unwinding into strands B and C. The signals sent from Cabin S5 will be shown at the origin of the B and C strands.

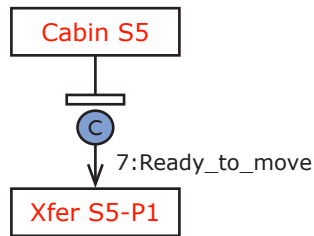
Terminal 

The terminal symbol indicates that a strand has terminated without unwinding into any further strands. Termination occurs because the activity in a dwell has completed without emitting outbound signals.



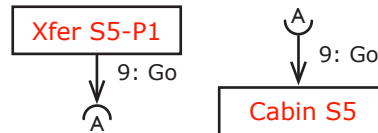
A test may require that one or more ancillary strands start and complete during a dwell of some base strand. The dwell in the base strand is duplicated and a weave symbol is placed in between the dwell symbols. The first dwell represents entry into a dwell and the emission of any signals prior to initiation of the ancillary strands. The second dwell represents activity or signal emission, if any, occurring after the ancillary strands terminate.

Thread chart notation glossary (continued)



Knot

A strand may reach a point where it cannot continue until some other strand sends a synchronizing signal. The knot symbol represents this tying together of strands with a line and hollow rectangle over the waited upon strand. In the example above, a base strand reaches the Cabin S5 dwell which completes its activity and waits for the 7:Ready_to_move signal to arrive from the C strand. The base strand then resumes by entering the Xfer S5-P1 object dwell.

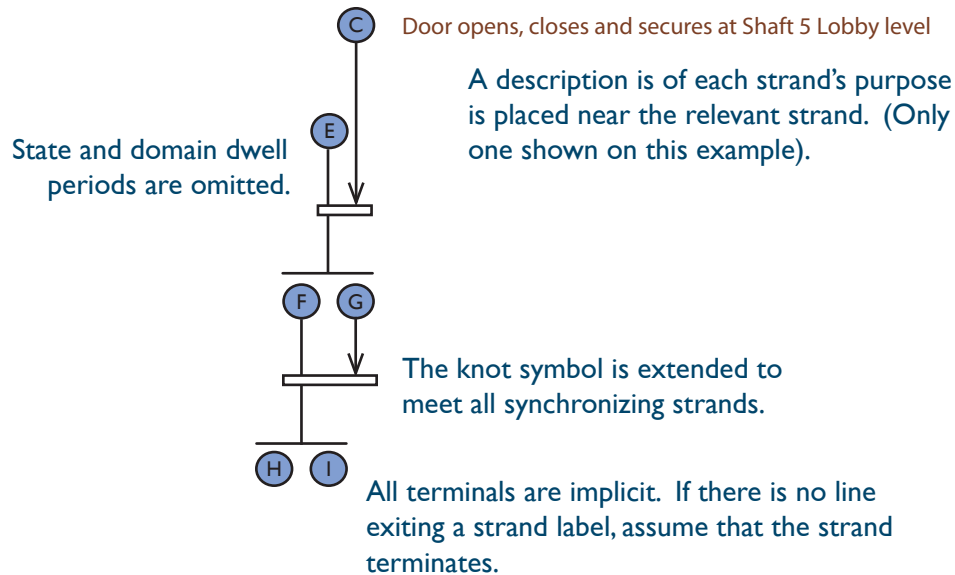


Page Connector

When a strand overflows the page boundary, you can wrap it around on the same page or on to another using matching page connector symbols. The continuing strand label is placed in the connector semicircle symbols.

Thread overview notation

A complex test represented by a thread chart may span multiple pages if all of the object and domain dwell periods are shown. A thread overview omits the dwell periods to concentrate on the overall flow and key points of synchronization. The symbols on the thread overview diagram are identical to those on the thread chart with the following differences:



In this excerpt from thread F1 we see that the E strand waits for the C strand to complete before continuing and unwinding into strands F and G. F then waits for G to complete before unwinding into strands H and I.

Elevator Application tests

Now we will run a couple of tests in the Elevator Application. The first test will be fairly simple and just take us through the processing of a single button push from inside the Cabin. This is enough to explore most of the key activities. Next we will test a combination of two presses and see what kind of concurrent havoc ensues.

In each test, we trace a thread as it unwinds, knots and eventually terminates in one or more strands. A trace begins with a statement of initial conditions and a thread overview diagram. A detailed narrative description takes us through the key activities in each domain and object dwell. Key synchronization and data access points are highlighted with an icon and explained in the notes section at the end of each test. A detailed thread chart is provided at the conclusion. The following symbols and conventions are used throughout the test descriptions.

Symbols used in narrative descriptions

A few symbols are scattered throughout the narrative descriptions to make them easy to follow. They are explained here.

Signals / Detected Events

Receive inbound signal from elsewhere

 <event_name>

Process internally generated signal

 <event_name>

Signal to self (forced transition)

Generate outbound signal

 <dest instance>

To object or external entity.



Time delayed signal received
(time event)

Activity Borders

Domain

 <domain name>

Marks activity beginning in a new domain.

Object

 <class name + instance id> <previous state>



Class icon

Marks activity beginning in a new object. The color of the object boundary matches the object's domain color.

Note icons

Synch note



Key point of synchronization. Explanations in numbered notes after end of the trace descriptions.

Data note



Key point involving the navigation of relationships or some aspect of data access.

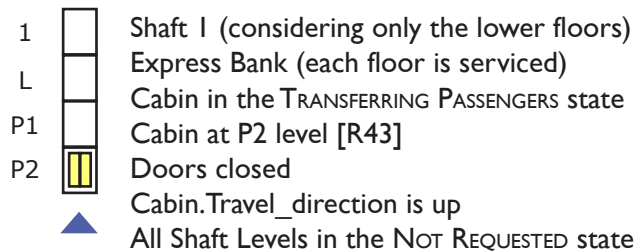
Thread S1: Single Stop Request

A passenger will enter the cabin at parking level 2, wait for the doors to close and then press the lobby (L) button. The cabin should go up to the lobby level and drop off the passenger.

Initial conditions

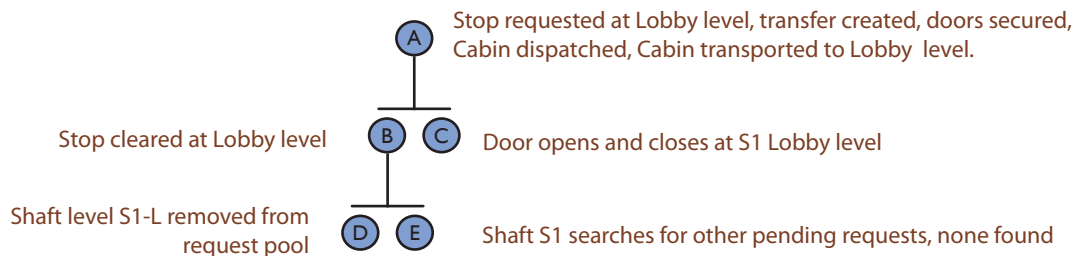
We assume the following initial conditions:

Shaft S1 in Bank "Express"



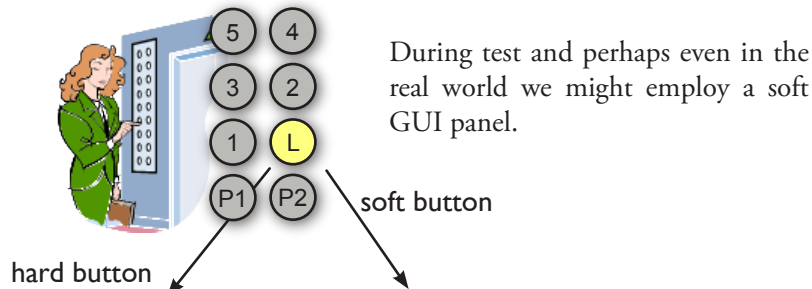
Thread overview diagram

Thread S1



Narrative description

- A A passenger enters the cabin at parking garage level 2 and pushes the L (lobby) button after the doors have closed.



Signal I/O

Signal is detected and a pre-registered UI bridge is invoked.

GUI Toolkit

Button widget changes state and a pre-registered UI bridge is invoked.

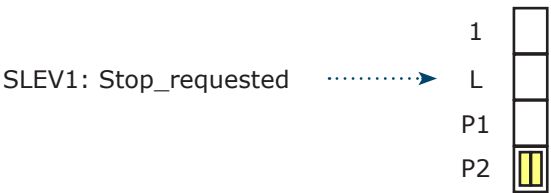
UI

A predefined UI Command is activated which bridges to the Elevator Application by invoking this domain function:

::UI_Stop_request (Shaft:"S1", Floor:"L")

Elevator Application

The UI bridge generates the SLEV1 signal to the S1-L destination Shaft Level.





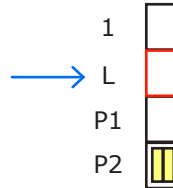
SLEV S1-L

NOT REQUESTED

The signal is received by the lobby Shaft Level.

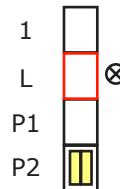
► SLEV1: Stop_requested Migrating to register stop

The Shaft Level migrates from Non_Requested to Requested.



▼ SLEV11: Migrated_for_stop Registering stop

The stop request is registered at the S1-L Shaft Level.



▼ SLEV9: Stop_registered Announcing new request

The S1 Shaft is notified that there is a new request pending.

Shaft1: Service_requested►



▼ SLEV5: Pending PENDING

Shaft Level S1-L waits to be serviced.



Shaft S1

WAITING FOR STOP
OR CALL REQUEST

► Shaft1: Service_requested Searching for best destination

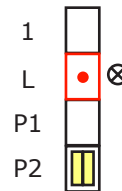
Using the search algorithm defined in tech note 6, the Shaft scans itself looking for the best Cabin destination. The newly registered stop request is selected since it is the only choice.

▼ Shaft4: Dest_selected (Dest_floor:"L", Dir:up) Verifying In Service

Before proceeding, we verify that the Shaft hasn't been taken out of service. It hasn't.

▼ Shaft9: Dest_selected (Dest_floor:"L", Dir:up) Creating or updating transfer

Since there is no existing Transfer, a new Transfer to the selected destination is created.



Xfer6: Created► ●

▼ Shaft3: Xfer_update_requested WAITING FOR STOP
OR CALL REQUEST

● Xfer S1-L

NEW



1

The Transfer was synchronously created in the NEW state by the Shaft.

► Xfer6: Created (Dir:up) Initializing

This Transfer's Call_in_progress attribute is set to false. This attribute is used later in the Transfer state chart to handle special behavior related to clearing floor calls. Since there is no floor call in this thread, we won't worry about it.

The UI is notified of the intended destination.

The Cabin's Travel_direction would be adjusted to point toward our destination but it's already been initialized to point up, so no change is necessary.

● ◄ Call_in_progress
is set to false.

A

● Xfer S1-L (continued)

We tell the Cabin that the destination has been set. The Cabin will let us know when it is ready to move.

Cabin1: New_transfer> 

▼ Xfer2: Initialized WAITING FOR SERVICE



2

We know that the Cabin will not move while we are in this state. This means that any potential destination changes can be handled without considering the transportation physics.



Cabin S1

TRANSFERRING PASSENGERS

The Door is opening and closing in this state so the Cabin cannot move yet. The Door must be put in a state where it ignores passenger requests to open.

► Cabin1: New_transfer Are we already there?

Before securing the Door we first check to see if we are already sitting at the desired destination. If that were the case, we would skip the transport phase and just report arrival. But we aren't, so we get ready to move.

▼ Cabin6: Prepare_to_move SECURING DOORS



3

We set the Door.Lock_requested to true and let the Cabin know.

Door2: Lock> 

Now we wait until the Door reports back that it is closed and ignoring passenger open requests.

A



Door S1

CLOSED

▶ Door2: Lock LOCKED

As directed, the Door stays closed and begins ignoring passenger open requests. It is now safe for the Cabin to move. The Lock_requested attribute is reset to false at this point.

Cabin3: Doors_secure 




Cabin S1

SECURING DOORS

▶ Cabin3: Doors_secure LOCKING TRANSFER



The Cabin is ready to move, but the Transfer might be changing the destination right now. So we need to alert the Transfer to finish up and let us know when to go.

4

Xfer7: Ready_to_move 



Xfer S1-L

WAITING FOR SERVICE

▶ Xfer7: Ready_to_move DISPATCHING CABIN

If we were in the middle of a pre-move Transfer update, we would first complete the edit. But we aren't, so we just tell the Cabin to go.

Cabin9: Go 

▼ Xfer4: Cabin_dispatched SERVICE IN PROGRESS

We wait here handling potential changes to the destination (if the Shaft finds somewhere better to stop along the way) until the Cabin arrives.



Cabin9: Go

....

REQUESTING TRANSPORT
TO DESTINATION

To crank up the motors and start moving we submit a request to the Transport domain.

```
if (TRAN::Goto_floor(Dest_floor:"L", Shaft_ID:"S1"))
```



5

A true/false value is returned by the Goto_floor service to indicate whether compliance is physically possible. If the Cabin is stationary, as it always is in this state, the return value should be true unless something is broken. Later, when the Cabin is moving, a false value will indicate that the requested destination has already passed or is too close for a safe stop.

The Transport domain has not yet been modeled so the bridge call is stubbed out as a temporary subsystem controlled using scaffolded domain functions such as ::TRANxx_Goto_floor. The name can be easily reformatted as a bridge call (TRAN::Goto_floor) when a real Transport domain replaces the stubbed subsystem.



Cabin11: Transport_in_progress MOVING

Transport

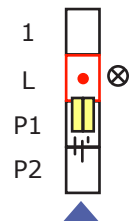
```
success = TRAN::Goto_floor(Dest_floor:"L", Shaft_ID:"S1")
```

This bridge call generates a signal in the Transport domain. Note that the signal is worded with Transport vocabulary.

Load n: Move (Stop:<stop_name>)

If the Transport domain is modeled with Executable UML, it might select a Load object corresponding to the Cabin object and a choose a preregistered Stop value corresponding to the destination Floor. A distance/momentum calculation would be performed to determine whether or not the move is possible. If it were not possible, a false value would be returned along with control of the strand. Otherwise, a true value would be returned, though the strand would continue in the Transport domain managing control feedback behavior and interacting through Signal I/O to command the motors. Once the Load makes it to the Stop, the following bridge call would be invoked to resume the strand in the Elevator Application.

```
EV::Move_complete(Load:self.ID)
```



A

Elevator Application

The EV::Move_complete(...) bridge call generates the Arrived_at_floor signal to the S1 Cabin.



Cabin S1

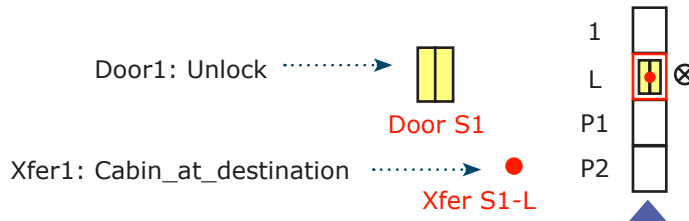
MOVING



Cabin4: Arrived_at_floor

..... TRANSFERRING
PASSENGERS

We've arrived at the lobby level so we tell the Door to open and notify the Transfer that delivery is complete.



There's nothing for the Cabin to do until another Transfer is created.

No immediate synchronization is necessary between the Door and Transfer instances, so they proceed concurrently.



B



Xfer S1-L

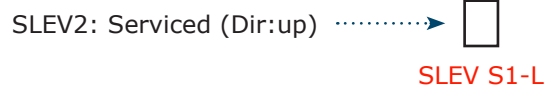
SERVICE IN PROGRESS



Xfer1: Cabin_at_destination

..... CLEARING REQUESTS

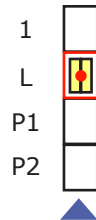
We notify the UI that the Cabin has arrived and tell the SLEV to clear its pending requests in the current direction. If there had been two calls up and down pending at this floor we would want to clear only the one in the Cabin's travel direction. In our case, there's only the lobby level stop request to clear.





► SLEV2: Serviced(Dir:up) CLEARING

We clear the stop request and if we had found a floor call in the up direction we would have cleared it, but we didn't. The UI is notified to clear the stop request (to turn the light off on the L button in the Cabin).



UI::Clear_stop_request (Floor_name:"L", Shaft_ID:"S1");

The UI update was issued by the Req_SLEV::Clear_stop operation.

The Transfer is notified that a stop / call or both has been cleared. If a call had been cleared, the direction would be returned. The Transfer needs to know the direction of any cleared call so that it can temporarily block the Shaft selection choice opposite of the cleared direction. No direction is assigned (null) when there is no floor call either way.

Xfer15: Serviced(Dir:null).....► ●
Xfer S1-L

We return to the PENDING state to wait until the Transfer has cleared itself out before migrating to Non Requested. (Otherwise we would break the R53 relationship and corrupt the Transfer). We must wait before migrating.

▼ SLEV5: Pending PENDING

► Xfer15: Serviced(Dir:null) Checking for hold

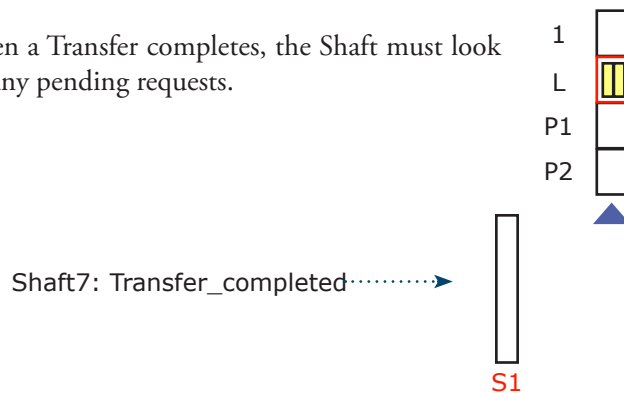
Since the returned direction is null, we know that only a stop request was cleared. We check to see if we are at the top or bottom of the Shaft and toggle the travel direction if necessary. Since we are in the middle of the shaft, we'll just leave the travel direction in the up direction. If a call had been cleared opposite the travel direction of the Cabin, we would have reversed the Cabin's direction.

Also, if a call had been cleared, we would need to block the selection of any destination opposite the travel direction until the Door had re-closed. Otherwise, a passenger expecting up service might be disappointed.

But none of this is necessary in this thread so we just delete this instance of Transfer.

▼ Xfer14: Delete Deleting

When a Transfer completes, the Shaft must look for any pending requests.



The requested Shaft Level has been waiting for the Transfer to unlink itself before migrating back to Non requested status, so we let it know we're gone.

SLEV6: Xfer_deleted ►

S1-L

This Transfer no longer exists.



No immediate synchronization is necessary between the Shaft Level and Shaft objects, so they proceed concurrently.

D



SLEV S1-L

PENDING



SLEV6: Xfer_deleted



Checking for
Unserviced Call

There is no pending call remaining at this Shaft Level so we migrate back to Non Requested. Had there been a pending call, we would have returned to the PENDING state and remained requested.



SLEV4: All_requests_cleared



NOT REQUESTED

E



Shaft S1

WAITING FOR STOP
OR CALL REQUEST



Shaft1: Service_requested



Searching for best
destination

The Shaft scans in search of another requested Shaft Level. None is found.



Shaft2: No_dest



WAITING FOR STOP
OR CALL REQUEST

The Shaft returns to the idle state and waits for another Shaft Level to become requested.



Door S1

LOCKED



▶ Door1: Unlock Unlocking

The Door open duration is loaded with the Bank's specified Passenger load time and the Door retry counter is reset to zero.



▼ Door13: Open_on_arrival OPENING

We decrement the max Retries counter and command the equipment to open. To do this we trigger an output signal via the Signal I/O domain.

SIO::Open_door (Shaft:"S1")

We also notify the UI that the door is now opening. A GUI, for example, could display an animation of an opening door at this point.

UI::Door_opening (Shaft_ID:"S1")



Signal I/O

The SIO domain will select a preregistered output signal corresponding to the bridge call name "Open_door" and the Shaft parameter value. This physical signal will then be loaded or modulated appropriately.

When the Door opens fully and trips a full open sensor, an input signal will be detected. The signal ID and detected value will be used to look up a preregistered Elevator Application bridge call. This bridge call will then be invoked.

For now the SIO stub simulates this activity and issues a Door3: Door_opened signal after a suitable time delay.

Signal I/O reports that the doors have fully opened. The appropriate event is triggered and sent to the S1 Door.

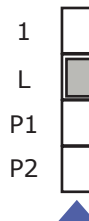


Door S1

OPENING

▶ Door3: Door_opened OPEN DELAY

Now the doors are fully opened and we arrange for a signal to be sent to the Door at the duration loaded into the Open_time attribute earlier. We also notify the UI of our progress.



UI::Door_opened (Shaft_ID:"S1");



▶ Door9: Time_to_close HOLDING OPEN

Even though our normal open delay has expired, it is possible that a passenger is pressing on the open button. The Held attribute will be set to true if this is the case. We want to hold the Door open until the button is released. In our thread, Held is false, so we generate the release signal to self and continue.

▼ Door11: Passenger_release CLOSING DOOR

Now we tell the physical doors to close and update the UI.

UI::Door_closing (Shaft_ID:"S1")

SIO::Close_door (Shaft:"S1")



The Door is commanded to close using the appropriate physical output.

At some point later, we detect either a door obstructed or a door fully closed signal. Assuming the latter, we trigger the corresponding signal in the Elevator Application domain.

Elevator Application

Signal I/O reports that the Door has fully closed. The appropriate signal is sent to the S1 Door.



Door S1

CLOSING DOOR

▶ Door4: Door_closed CLOSED

We check to see if a lock has been requested. Since there are no requested Shaft Levels and there is no active Transfer, we can safely assume that the Lock_requested value of this Door is false. Otherwise we would advance to the LOCKED state and ignore passenger open requests.

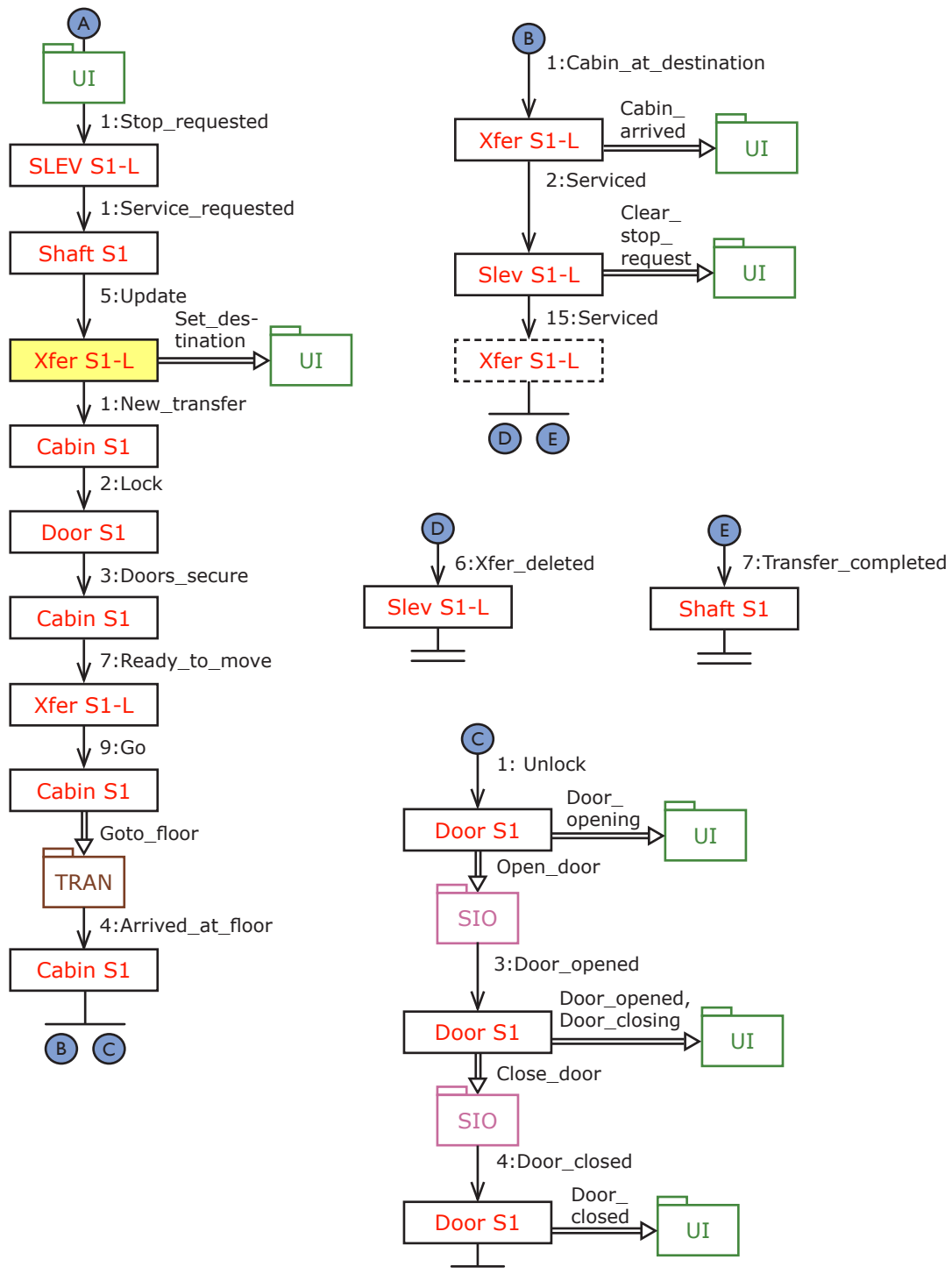
A Transfer might be waiting for the Door to close the first time after arrival. This is because the Transfer is blocking the Shaft from selecting lower levels when the Cabin is answering an up call or vice versa. No Transfer exists so we just wait here in the CLOSED state until the Cabin wants to move, a passenger tries to open the Door with the open button or a passenger makes a request at this floor.



End of the S1 Thread

All strands have terminated, so this thread is complete. Here's a synopsis of what happened. A passenger in a closed Cabin pressed the lobby (L) button. A stop request was created and the S1 Shaft was prodded to look for work. The S1 Shaft found only the one stop request, so it created a transfer to the S1-L Shaft Level. This Transfer directed the Cabin to move to this destination. After verifying that its Door was closed and ignoring user open requests, the Cabin ascended to the lobby level, opened and then closed its Door.

Thread chart S1: Single stop request





- 1 The Transfer must be created synchronously. It's tempting to signal the Transfer to create and link itself. But there is no guarantee that the Transfer will be fully linked on R53 before the Shaft returns to the **CREATING OR UPDATING TRANSFER** state on a subsequent scan. If this were to happen, the Shaft would see an incomplete and possibly non-existent R53 link and might try to create an illegal second Transfer.
- 2 What happens if the Transfer is redirected to a new destination before the Cabin has been told to move? This is handled by simply updating the R53 link to the new destination Shaft Level. As long as we know that no Transport request has been issued - which must be the case in this state - we can make the switch without having to synchronize with the Cabin. So this state marks the period of time where we know we can make a no-hassle destination swap.
- 3 The Cabin cannot move unless the Door is not only closed, but in a state where user open requests will be ignored. This means that the Cabin must submit a lock request and then wait for the Door to confirm that it is waiting in the locked state.

When the Cabin submits this request, the Door is either closed and waiting for a lock request, or it is busy in some other state. In the **CLOSED** state the Door can react immediately to the lock request by advancing to the **LOCKED** state. In any other state the Door must somehow “save” the signal and see if it was sent upon entering the Closed state. How do we “save” the signal?

We use the signal-set-check collaboration pattern (see Part 9 - Collaboration Pattern 17). The Cabin simultaneously sends a lock request signal and sets the Door's Lock_request attribute to “true”. If the Door is busy when the Lock signal arrives, the signal is ignored. That's okay since the Lock_request attribute is checked upon entry to the Door's **CLOSED** state. If true, the Door proceeds to the Locked state and reports back to the Cabin.

The Lock_requested attribute is reset by the Door on arrival.

- 4 We don't want the Cabin to start moving toward the current destination while the Transfer is swapping destinations. So the Cabin must first report to the Transfer that it is ready to move. The Transfer can now complete any destination swap that might be in process and then advance to a safe state.
- 5 If the Cabin is moving quickly toward a Floor, and this Floor is suddenly requested, we might want to swap the current Transfer destination and stop there. But if the distance is too short and the Cabin is moving too fast, we can't. The Transport domain has all the data it needs to make this computation. So *when the Cabin is in motion* we must somehow consult the Transport domain before committing to a closer destination.

First let's consider a scheme riddled with synchronization vulnerabilities. We ask Transport if it is possible to stop at a given Floor, and with luck Transport responds in the affirmative. So we have the Transfer swap to the closer destination and then have the Cabin request the new stop from Transport. Okay, what could possibly go wrong with that scheme?

What if, in the period of time between the request affirmation and the command to go, it turns out that the Cabin is now too close to stop? Imagine how awful it will be to try and swap back to the original destination and then find out that we are moving too fast to stop there either! We need a foolproof scheme to avoid any chaos.

Instead we replace the ask-and-command scheme with a single `Goto_floor` command that will immediately return either true or false. False means that the requested destination is too close for a safe stop. True means that not only is the stop achievable, it is automatically the new drive-to destination in the Transport domain. *After* a true value is returned, we update the Transfer – never before.

To summarize, a potential change in destination is first suggested by the Shaft to the active Transfer. If the Cabin is already in motion, the Transfer asks the Cabin to *attempt* a destination change. If successful, (true returned) the Cabin will notify the Transfer which will update accordingly. Otherwise, the Cabin will continue as previously commanded and no change will have been made to the current destination of the Transfer.

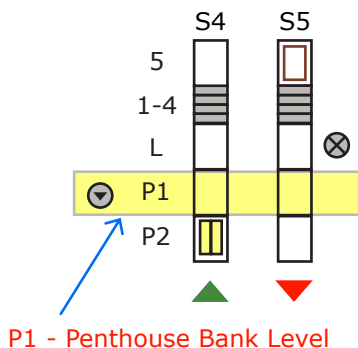
Thread F1: Floor Call with Stop Request

A passenger at the P1 parking level presses the down call button.
While the Cabin in Shaft S5 is on its way to respond, a passenger
in the S5 Cabin presses the L button.

Initial conditions

We assume the following initial conditions:

Two Shafts S4 and S5 in Bank "Penthouse"

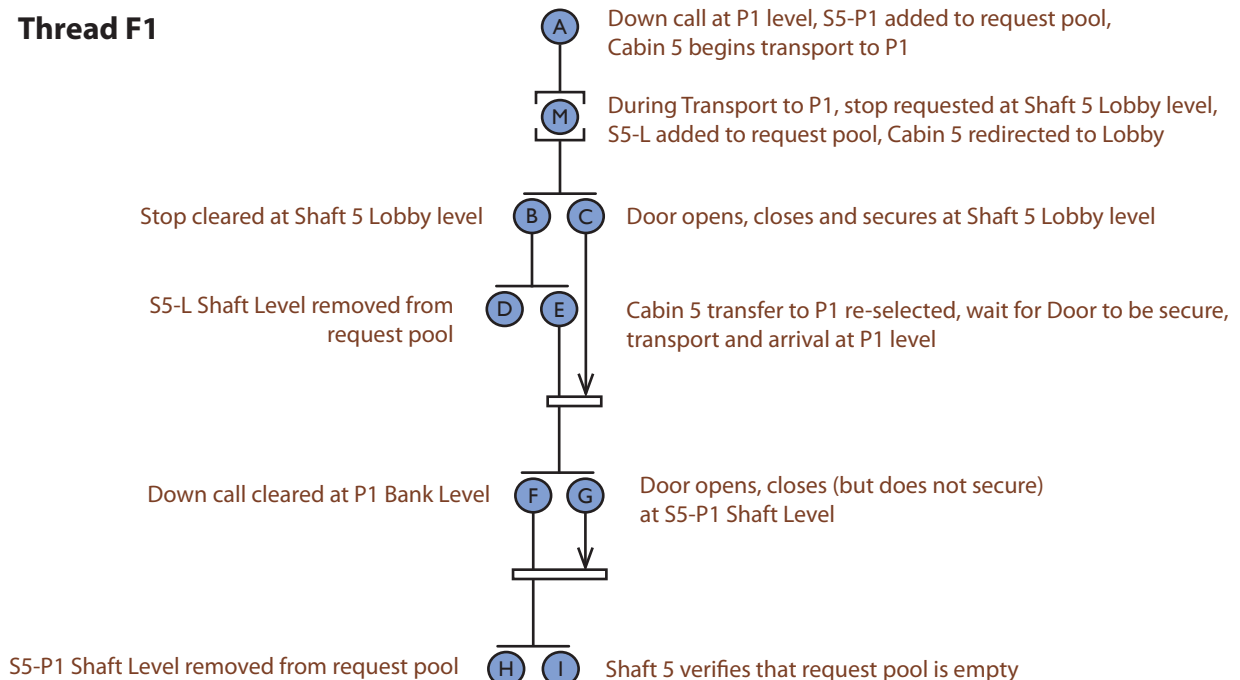


Penthouse Bank (Floors 1-4 not serviced)
Each Cabin is in the TRANSFERRING PASSENGERS state
Door S5 is OPEN and Door S5 is CLOSED
Cabin S4 at P1 level and S5 at 5 level [R43]
Cabin.Travel_direction S4 is up, S5 is down
All Shaft Levels in the NOT REQUESTED state

⊖ ⊗ These two requests are grayed out since they won't be registered until later.

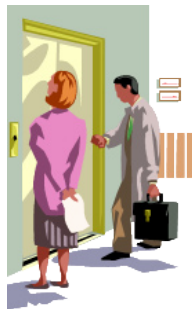
Thread overview diagram

Thread F1



Narrative description of thread F1

- Ⓐ A passenger at the P1 parking level presses the down call button.



P1



During test and perhaps even in the real world we might employ a soft GUI panel.

hard button

soft button

Signal I/O

Signal is detected and a pre-registered UI bridge is invoked.

GUI Toolkit

Button widget changes state and a pre-registered UI bridge is invoked.

UI

A predefined UI Command is activated which bridges to the Elevator Application by invoking this domain function:

```
::UI_Floor_call (Bank:"Penthouse", Floor:"P1", Dir:"down")
```

Elevator Application

The UI bridge selects the Penthouse-P1 Bank Level and invokes its Call_cabin operation. Since the Bank Level does not have an interesting lifecycle it has no statechart. Consequently, no signals/events are defined for the Bank Level class.

Call_cabin (Dir:down)



Penthouse-P1

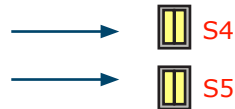
The Call_cabin operation is polymorphically routed to the Middle BLEV. The Middle BLEV must now select the Shaft likely to service the floor call quickest within the Bank.

```
if (self_BLEV.Choose_shaft(OUT_Shaft:"", Dir:down))
```

Choose_shaft(...)

We find the in service Cabin with the shortest travel delay to this Bank Level going in the correct direction. So we tell each Cabin in our Bank to estimate its travel delay.

```
cab_delay = Estimate_travel_delay()
```




Cabin S4

Estimate travel delay ()

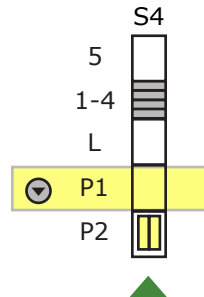
The distance and number of actual and potential pending requests between the Cabin and the destination determine the estimated travel delay. The algorithm is explained in detail in tech note 6. It is required reading to understand how travel delays are estimated.

The shape of the path to the destination must first be determined, starting from the nearest Bank Level.



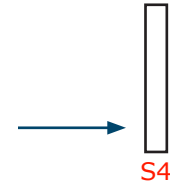
1

```
self.Nearest_BLEV(Dir:up)
↳ returns Penthouse-P1
```



Now we count the number of pending stops from the current location to the calling floor.

```
Count_stops (Fwd_only:true, Start_floor:P2, End_floor:
P1, Include_start:true, Include_end:true)
↳ returns 0
```





Shaft S4

Count stops (...)

Counting from the specified start floor to the end floor, including or excluding the start and end floors as specified, the stops are counted. A Shaft Level counts as a stop if it has a pending stop request or a floor call in the start-end direction. A Shaft Level with only an opposite call pending are is not counted if Fwd_only is specified. This is the case in our example. Otherwise these are counted as well. Again, all this is explained in tech note 6.

In our case, zero is returned.



Cabin S4

Estimate travel delay ()

Stops are counted for the remaining leg—calling floor to upward turn around and back to the calling floor.

```
self.Nearest_BLEV(Dir:up)
↳ returns Penthouse-P1
```

```
Count_stops (Fwd_only:false, Start_floor:P1, End_floor:5,
             Include_start:false, Include_end:true)
↳ returns 0
```

Again zero is returned. The stops are totaled for all legs with a grand total of zero which means that there is no delay due to stops en route. There is a delay due to distance traveled. This is computed, added to the stops delay and returned as the estimated travel delay for this Cabin.



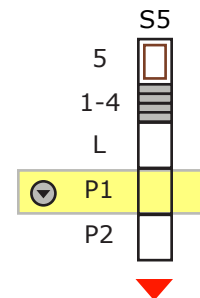
Cabin S5

Estimate travel delay ()

The same sequence of function invocations is carried out for the S5 Cabin. Since the Cabin is already pointed in the direction of the floor call, only one call to Count_stops is required. We count from the current floor to the calling floor.

```
self.Nearest_BLEV (Dir:down)
↳ returns Penthouse-5
```

```
Count_stops (Fwd_only:true, Start_floor:5, End_floor:
             P1, Include_start:true, Include_end:false)
↳ returns 0
```



Cabin S4 is closer to the calling floor, but it is pointed the wrong direction and could potentially run all the way to the top and turn around, catching any last minute stops along the way. Cabin S5 could get there sooner since it is already pointed in the correct direction and there is less possibility of stop delays en route. Keep in mind that the travel delay estimate algorithm does not predict the future. It simply assumes the worst case all of the time.

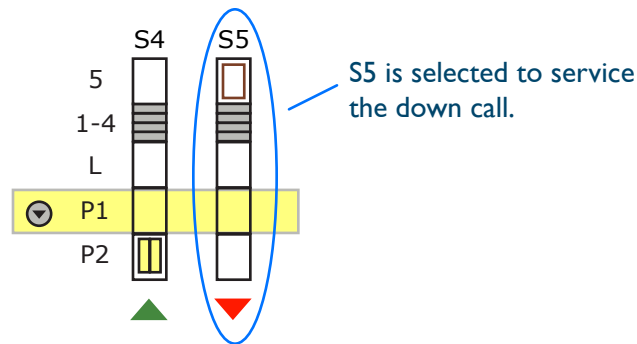
The estimated travel delay is computed for each Cabin and then the lowest value is selected. The corresponding Shaft ID is returned.

```
if (self_BLEV.Choose_shaft(OUT_Shaft:"", Dir:down))
```

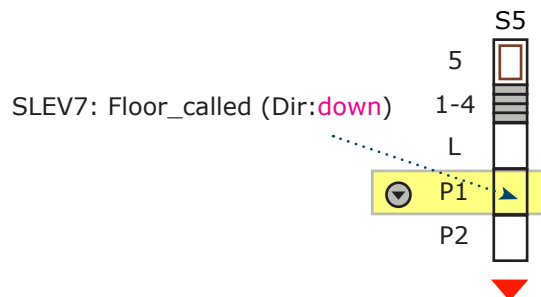
Call_cabin(Dir)

↳ returns true

↓ S5 returned by reference



Now we grab the S5-P1 Shaft Level and tell it to register a floor call in the down direction. Note that while the floor call was initially directed at a Bank Level, it is now attributed to a Shaft Level and will be assigned to the Shaft request pool.





SLEV S5-P1

NOT REQUESTED



SLEV7: Floor_called (Dir:down)

Migrating to register call

The Shaft Level migrates from Non_Requested to Requested.

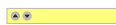


SLEV10: Migrated_for_call(Dir:down)

Registering call

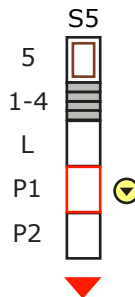
Now that the Shaft Level has migrated to Requested, the Bank Level is instructed to link itself up so that the floor call is registered on the Shaft's request pool. Note that this behavior is a bit more complex than the registration of a stop request. With a stop request you simply set the stop request attribute to true. But a floor call involves more information. You must remember the direction of the call and this is done with a link to the calling Bank Level.

Set_call (Shaft:S5, Dir:down)



Mid_BLEV Penthouse-L

The Middle Bank Level links itself to the S5-P1 Shaft Level.



Set_call (S5, down)



SLEV S5-P1

Registering call



SLEV8: Call_registered

Announcing new request

Shaft1: Service_requested



SLEV5: Pending

PENDING

Shaft Level S5-P1 waits to be serviced.



Shaft S5

WAITING FOR STOP
OR CALL REQUEST

► Shaft1: Service_requested Searching for best destination

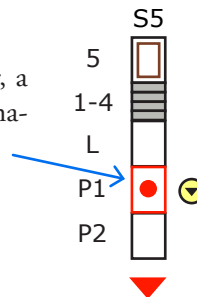
Using the search algorithm defined in tech note 1, the Shaft scans itself looking for the best Cabin destination. The newly registered floor call is selected since it is the only choice.

▼ Shaft4: Dest_selected (Dest_floor:"P1", Dir:down) Verifying In Service

Before proceeding, we verify that the Shaft hasn't been taken out of service. It hasn't.

▼ Shaft9: Dest_selected (Dest_floor:"P1", Dir:down) Creating or updating transfer

Since there is no existing Transfer, a new Transfer to the selected destination is created.



▼ Shaft3: Xfer_update_requested WAITING FOR STOP
OR CALL REQUEST

● Xfer S5-P1

NEW

The Transfer was synchronously created in the NEW state by the Shaft.

► Xfer6: Created(Dir:down) Initializing

This Transfer's Call_in_progress attribute is set to false.
We'll use it when the Cabin arrives at the destination

The UI is notified of the intended destination.

UI::Set_destination (Shaft_ID:"S5", Floor_name:"P1")

The Cabin's Travel_direction is already pointed down, so no change is necessary.

● ◀ Call_in_progress
is set to false.

A

● Xfer S5-P1 (continued)

We tell the Cabin that the destination has been set. The Cabin will let us know when it is ready to move.

Cabin1: New_transfer ➡ 

▼ Xfer2: Initialized WAITING FOR SERVICE



Cabin S5

TRANSFERRING PASSENGERS

The Door is now open so the Cabin cannot move. The Door must be close and enter a state where it ignores passenger requests to open.

▶ Cabin1: New_transfer Are we already there?

Before securing the Door we first check to see if we are already sitting at the desired destination. That would save a lot of trouble! We aren't, so we get ready to move.

▼ Cabin6: Prepare_to_move SECURING DOORS

We set the Door.Lock_requested to true and let the Cabin know.

Door2: Lock ➡  Lock_request attribute set to true.

Now we wait until the Door reports back that it is closed and ignoring passenger open requests.



Door S5

OPEN DELAY

▶ Door2: Lock ✖

The lock signal is ignored! The Door is open and it will stay that way until the open time expires and the passenger is not pressing down on the open button. So, at this point, there is no response to the lock signal. Good thing we set the Lock_requested attribute to "remember" the signal!



▶ Door9: Time_to_close HOLDING OPEN

A



Door S5 (continued)

Even though our normal open delay has expired, it is possible that a passenger is pressing on the open button. The Held attribute will be set to true if this is the case. We want to hold the Door open until the button is released. In our case, Held is false, so we generate the release signal to self and continue.

▼ Door11: Passenger_release CLOSING Door

Now we tell the physical doors to close and update the UI.

UI::Door_closing (Shaft_ID:"S5")

SIO::Close_door (Shaft:"S5")

Signal I/O

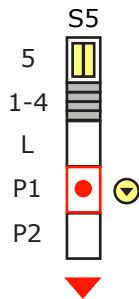
The Door is commanded to close using the appropriate physical output.

At some point later, we detect either a door obstructed or a door fully closed signal. Assuming the latter, we trigger the corresponding signal in the Elevator Application domain.



Door S5

CLOSING Door



At this point we check the Lock_requested attribute and find it is set to "true". So we proceed to the LOCKED state.



▶ Door4: Door_closed CLOSED

▼ Door2: Lock LOCKED

The Door now ignores passenger open requests. It is safe for the Cabin to move.

Cabin3: Doors_secure▶



UI::Door_closed (Shaft_ID:"S5")



Cabin S5

SECURING DOORS

▶ Cabin3: Doors_secure LOCKING TRANSFER

The Cabin is ready to move, but the Transfer might be changing the destination right now. So we need to alert the Transfer to finish up and let us know when to go.

Xfer7: Ready_to_move▶ ●



Xfer S5-P1

WAITING FOR SERVICE

▶ Xfer7: Ready_to_move DISPATCHING CABIN

If we were in the middle of a pre-move Transfer update, we would first complete the edit. But we aren't, so we just tell the Cabin to go.

Cabin9: Go▶

▼ Xfer4: Cabin_dispatched SERVICE IN PROGRESS

We wait here handling potential changes to the destination (if the Shaft finds somewhere better to stop along the way) until the Cabin arrives.



Cabin S5

LOCKING TRANSFER

▶ Cabin9: Go REQUESTING TRANSPORT
TO DESTINATION

To crank up the motors and start moving we submit a request to the Transport domain.

```
if (TRAN::Goto_floor(Dest_floor:"P1", Shaft_ID:"S5"))
```

```
  ↳ returns true
```

Currently stubbed out as ::TRANxx_Goto_floor.

▼ Cabin11: Transport_in_progress MOVING

A

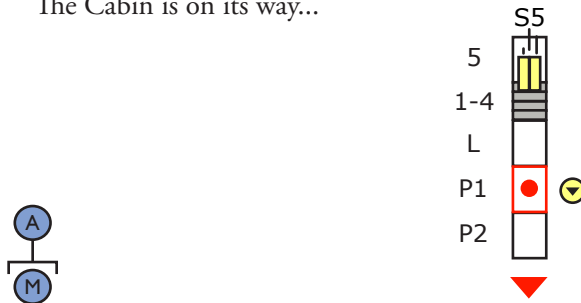
Transport

```
success = TRAN::Goto_floor(Dest_floor:"P1", Shaft_ID:"S5")
```

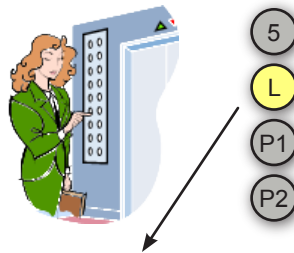
This bridge call generates a signal in the Transport domain. Note that the signal is worded with Transport vocabulary.

Load n: Move (Stop:<stop_name>)

The Cabin is on its way...



While the Cabin S5 is in motion downward, a passenger in the cabin pushes the first floor button.



M

UI

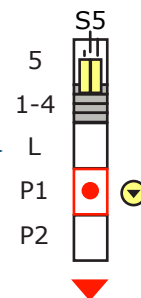
A predefined UI Command is activated which bridges to the Elevator Application by invoking this domain function:

```
::UI_Stop_request (Shaft:"S5", Floor:"L")
```

Elevator Application

The UI bridge generates the SLEV1 signal to the S5-L destination Shaft Level.

SLEV1: Stop_requested

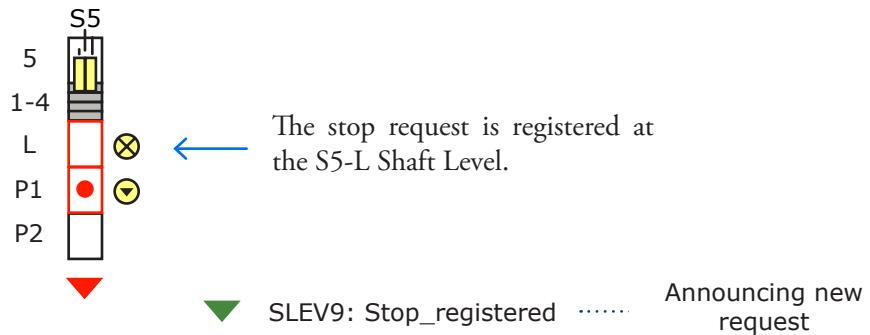
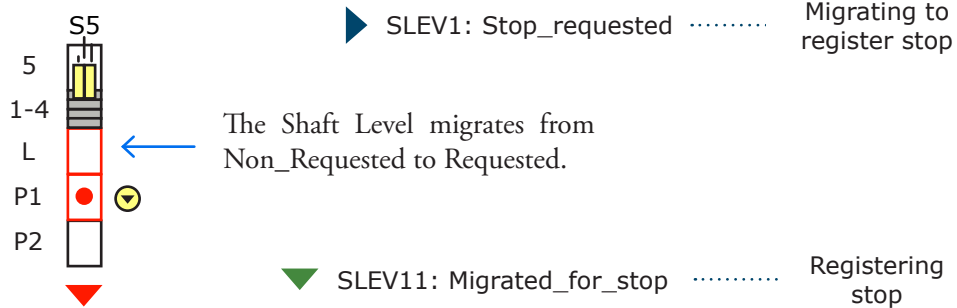




SLEV S5-L

NOT REQUESTED

The signal is received by the lobby Shaft Level.



The S5 Shaft is notified that there is a new request pending.

Shaft1: Service_requested



SLEV5: Pending PENDING

Shaft Level S5-L waits to be serviced.

Shaft S5

WAITING FOR STOP
OR CALL REQUEST▶ Shaft1: Service_requested Searching for Best
Destination

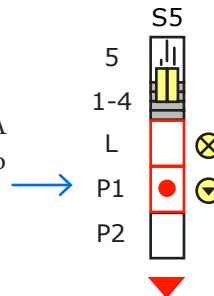
The Shaft search algorithm sees that the new stop request is the closest possible destination in the current direction. So it is tentatively selected.

▼ Shaft4: Dest_selected (Dest_floor:"L", Dir:down) Verifying In Service

Before proceeding, we verify that the Shaft hasn't been taken out of service. It hasn't.

▼ Shaft9: Dest_selected (Dest_floor:"L", Dir:down) Creating or Updating
Transfer

There is already an active Transfer! A change request must be submitted to that Transfer.

▼ Shaft3: Xfer_update_requested WAITING FOR STOP
OR CALL REQUEST

● Xfer S5-P1

SERVICE IN PROGRESS

The Transfer to the Lobby floor is still active. But a change request has been received from Shaft S5.

▶ Xfer5: Update(Dest_floor:"L") Requesting Change
to Destination

We ask the Cabin to *attempt* re-routing to the new destination floor.

Cabin10: Change_destination(New_dest:"L")▶ 

Having issued the request, we continue in our previous state. No actual change will be made unless the Cabin confirms that it is going somewhere else.

▼ Xfer3: Update_requested SERVICE IN PROGRESS



► Cabin10: Change_destination (New_dest:"L") Changing Destination

We pass this request to the Transport domain to see if a stop at the tentative destination is physically possible.

```
if (TRAN::Goto_floor(Dest_floor:"L", Shaft_ID:"S5"))
```

Currently stubbed out as ::TRANxx_Goto_floor.

Transport

```
success = TRAN::Goto_floor(Dest_floor:"L", Shaft_ID:"S5")
```

The stubbed out version of this function always updates the stop destination and returns true. (When the new destination is very near the current Cabin position the Cabin icon may jump back a couple increments. No point fixing this until we build the real thing).

In the real Transport domain, a calculation or lookup would be performed to determine whether or not the stop can be accommodated given safety considerations, g-force limits, equipment tolerances, etc.

Since we are working with the Transport stub, a true value is returned.



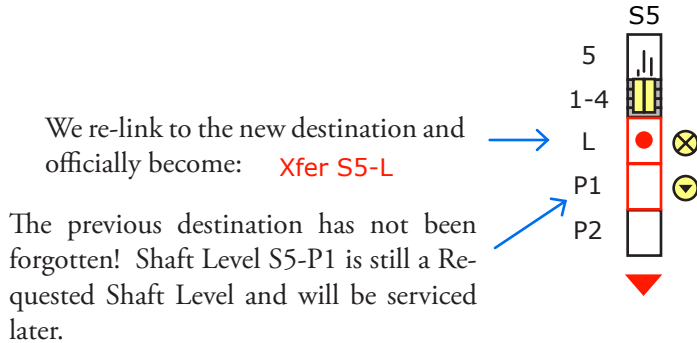
```
if (TRAN::Goto_floor(Dest_floor:"L", Shaft_ID:"S5"))
```

True was returned, so we notify the active Transfer that the change has succeeded and return to the MOVING state.

```
Xfer10: Cabin_redirected (Updated_dest:"L") .....►●
                                             Xfer S5-P1
```

▼ Cabin11: Transport _in_progress MOVING

► Xfer10: Cabin_redirected (Updated_dest:"L") Updating - Cabin Moving



UI::Set_destination (Shaft_ID:"S5", Floor_name:"L")

▼ Xfer9: Update_succeeded SERVICE IN PROGRESS



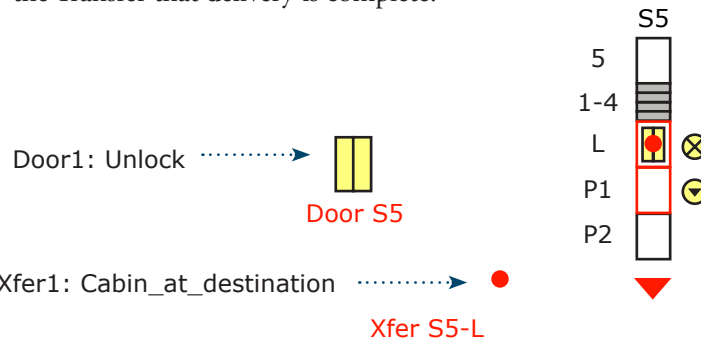
Cabin S5

MOVING

Motion stops and Transport announces the arrival of the Cabin.

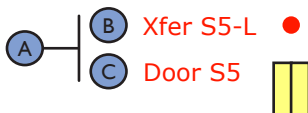
► Cabin4: Arrived_at_floor TRANSFERRING PASSENGERS

We've arrived at the lobby level so we tell the Door to open and notify the Transfer that delivery is complete.



There's nothing for the Cabin to do until a new Transfer is created.

No immediate synchronization is necessary between the Door and Transfer instances, so they proceed concurrently.



► Xfer1: Cabin_at_destination CLEARING REQUESTS

We notify the UI that the Cabin has arrived and tell the SLEV to clear its pending requests in the current direction. In our case, there's only the lobby level stop request to clear.

UI::Cabin_arrived (Shaft_ID:"S5")

SLEV2: Serviced(Dir:up) ► 
SLEV S5-L

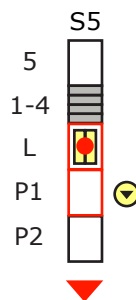
 SLEV S5-L PENDING

► SLEV2: Serviced(Dir:up) CLEARING

We clear the stop request and if we had found a floor call in the up direction we would have cleared it, but we didn't. The UI is notified to clear the stop request (to turn the light off on the L button in the Cabin).

UI::Clear_stop_request (Floor_name:"L", Shaft_ID:"S5")

The UI update was issued by the Req_SLEV::Clear_stop operation.



The Transfer is notified that the stop request has been cleared. No direction is assigned (null) since no floor call was cleared at this level.

Xfer15: Serviced(Dir:null) ► ●
Xfer S5-L

We return to the PENDING state to wait until the Transfer has cleared itself out before migrating to Non Requested.

▼ SLEV5: Pending PENDING

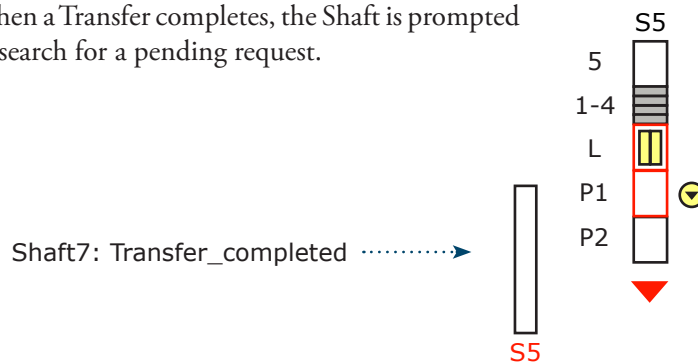
► Xfer15: Serviced(Dir:null) Checking for Hold

Since the Dir parameter value is null, we know that we did not service a floor call, so there is no need to change travel direction. We also check to see if we have reached the top or bottom of the Shaft. This condition necessitates a travel direction reversal, but we are somewhere in the middle, so we just leave the travel direction in the down direction.

We unlink and delete ourself.

▼ Xfer14: Delete Deleting

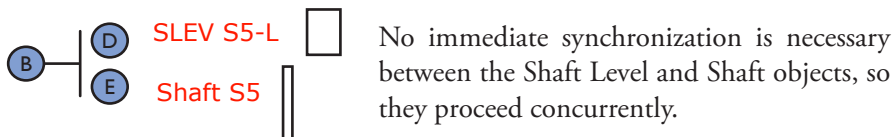
When a Transfer completes, the Shaft is prompted to search for a pending request.



The requested Shaft Level has been waiting for the Transfer to unlink itself before migrating back to Non Requested status. So we let the Shaft Level know we're gone.

SLEV6: Xfer_deleted ► 
S5-L

This Transfer no longer exists.



D

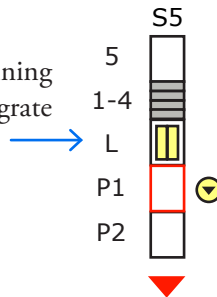


SLEV S5-L

PENDING

► SLEV6: Xfer_deleted Checking for Unserved Call

There is no pending call remaining at this Shaft Level so we migrate back to Non Requested.



▼ SLEV4: All_requests_cleared NOT REQUESTED

E



Shaft S5

WAITING FOR STOP OR CALL REQUEST

► Shaft7: Transfer_completed Searching for best destination

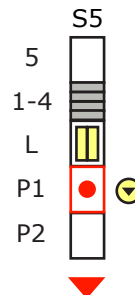
The Shaft scans in search of another Requested Shaft Level. There is only one left, so it is selected.

▼ Shaft4: Dest_selected (Dest_floor:"P1", Dir:up) Verifying In Service

Before proceeding, we verify that the Shaft hasn't been taken out of service. It hasn't.

▼ Shaft9: Dest_selected (Dest_floor:"P1", Dir:down) Creating or updating transfer

Since there is no existing Transfer, a new Transfer to the selected destination is created.



Xfer6: Created ► ●

▼ Shaft3: Xfer_update_requested WAITING FOR STOP OR CALL REQUEST

The Transfer was synchronously created in the NEW state by the Shaft.

► Xfer6: Created(Dir:down) Initializing

This Transfer's Call_in_progress attribute is set to false.
We'll use it when the Cabin arrives at the destination. ● ◆

The UI is notified of the intended destination.

UI::Set_destination (Shaft_ID:"S5", Floor_name:"P1")

The Cabin's Travel_direction is already pointed down, so no change is necessary.

We tell the Cabin that the destination has been set.

Cabin1: New_transfer ► 

▼ Xfer2: Initialized WAITING FOR SERVICE



► Cabin1: New_transfer Are we already there?

Before securing the Door we first check to see if we are already sitting at the desired destination. If that were the case, we would skip the transport phase and just report arrival. But we aren't, so we get ready to move.

▼ Cabin6: Prepare_to_move SECURING DOORS

We set the Door.Lock_requested to true and let the Cabin know.

Door2: Lock ► 



Now we wait until the Door reports back that it is closed and ignoring passenger open requests.



At this point two signals are pending, Unlock and Lock, both generated by Cabin S5. The Executable UML synchronization rules state that signals from the same sender are processed in the sending order. So we experience the Unlock event first.

► Door1: Unlock Unlocking

The Door open duration is loaded with the Bank's specified Passenger load time and the Door retry counter is reset to zero.

Can we process the Lock signal now? No! Since we send a signal to self here, that gets priority, so we move on. The Lock signal is not lost, since it has not yet been consumed.

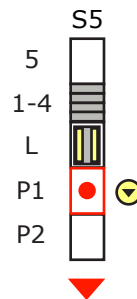
▼ Door13: Open_on_arrival OPENING

We decrement the max Retries counter and command the equipment to open.

SIO::Open_door (Shaft:"S5")

We also notify the UI that the door is now opening.

UI::Door_opening (Shaft_ID:"S5")



Now that we've completed the activity in the Opening state and are waiting to hear back from SIO, with no signal-to-self pending, we finally have an opportunity to consume the Lock signal.

► Door2: Lock ✖

And, as you can verify from the Door state table, the signal is ignored. But we've got the Lock_requested set so the information is saved anyway.

The SIO domain opens the Door and also triggers a Door3: Opened event when the physical Door has fully opened.

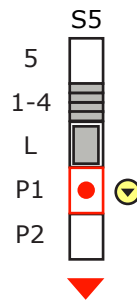


Door S5

OPENING

▶ Door3: Door_opened OPEN DELAY

Now the Door is fully opened and we arrange for a signal to be sent to us at the duration loaded into the Open_time attribute. We also notify the UI of our progress.



UI::Door_opened (Shaft_ID:"S5");



▶ Door9: Time_to_close HOLDING OPEN

Even though our normal open delay has expired, it is possible that a passenger is still pressing on the open button. The Held attribute will be set to true if this is the case. We want to hold the Door open until the button is released. In our case, Held is false, so we generate the release signal to self and continue.

▼ Door11: Passenger_release CLOSING DOOR

Now we tell the physical Door to close and update the UI.

UI::Door_closing (Shaft_ID:"S5")

SIO::Close_door (Shaft:"S5")

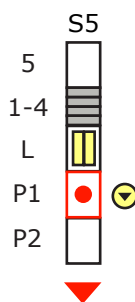
The Door is commanded to close using the appropriate physical output.

At some point later, we detect either a door obstructed or a door fully closed signal. Assuming the latter, we trigger the corresponding signal in the Elevator Application domain.



Door S5

CLOSING DOOR



At this point we check the Lock_requested attribute and find it is set to "true". So we proceed to the LOCKED state.



Door4: Door_closed CLOSED

Door2: Lock LOCKED

The Door now ignores passenger open requests. It is safe for the Cabin to move.

Cabin3: Doors_secure

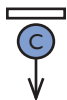
UI::Door_closed (Shaft_ID:"S5")



Xfer S5-P1

WAITING FOR SERVICE

Xfer7: Ready_to_move DISPATCHING CABIN

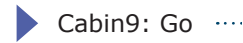


Now that the C strand has terminated, we know that the Door is secure so we can resume the E strand.

We aren't in the middle of a swap, so we just tell the Cabin to go.

Cabin9: Go

Xfer4: Cabin_dispatched SERVICE IN PROGRESS



Cabin9: Go

....

REQUESTING TRANSPORT
TO DESTINATION

We submit a request to the Transport domain to crank up the motors and start moving.

```
if (TRAN::Goto_floor(Dest_floor:"P1", Shaft_ID:"S5"))
```

Currently stubbed out as ::TRANxx_Goto_floor.



Cabin11: Transport_in_progress MOVING

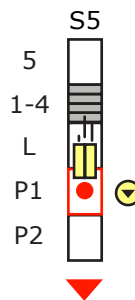
Transport

```
success = TRAN::Goto_floor(Dest_floor:"P1", Shaft_ID:"S5")
```

This bridge call generates a signal in the Transport domain. Note that the signal is worded with Transport vocabulary.

```
Load n: Move (Stop:<stop_name>)
```

The Cabin is on its way...

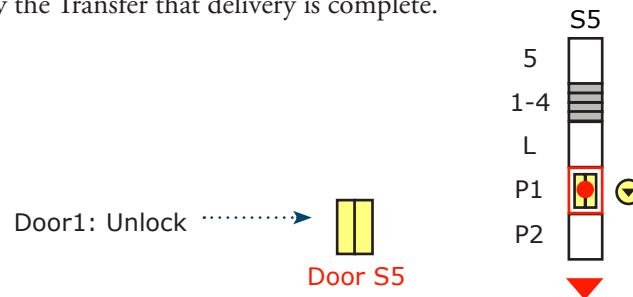




Motion stops and Transport announces the arrival of the Cabin.

► Cabin4: Arrived_at_floor TRANSFERRING PASSENGERS

We've arrived at the lobby level so we tell the Door to open and notify the Transfer that delivery is complete.



Xfer1: Cabin_at_destination ► Xfer S5-P1

There's nothing for the Cabin to do until another Transfer is created.

No immediate synchronization is necessary between the Door and Transfer instances, so they proceed concurrently.



► Xfer1: Cabin_at_destination CLEARING REQUESTS

We notify the UI that the Cabin has arrived and tell the Shaft Level to clear its pending requests in the current direction. In our case, there's only the lobby level stop request to clear.

SLEV2: Serviced(Dir:down) ► 
SLEV S5-P1

 SLEV S5-P1 PENDING

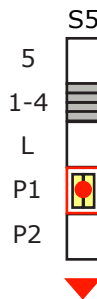
► SLEV2: Serviced(Dir:down) CLEARING

We clear any stop request or call in the down direction. So we clear our down call at P1. The UI is notified to clear the call (to turn the down arrow light off at the P1 Bank Level).

UI::Clear_floor_call (Bank_name:"Penthouse", Floor_name:"P1", Dir:"down");



This UI update was issued in the Mid_BLEV::Clear_call(Dir) operation.



The Transfer is notified that a floor call has been cleared in the down direction.

Xfer15: Serviced(Dir:down) ► ●
Xfer S5-P1

We return to the PENDING state to wait until the Transfer has cleared itself out before migrating to Non Requested.

▼ SLEV5: Pending PENDING

▶ Xfer15: Serviced(Dir:down) Checking for Hold

Since the returned direction is down, we know that a floor call has been cleared. This means that the Transfer won't delete itself immediately. Instead, we set our Call_requested to true and wait.

- ▼ Call requested true and opposite of Cabin's travel direction is blocked from destination selection.

This restricts the Shaft to select only a floor ahead of the Cabin in the current travel direction. So if someone jumps into the Cabin and quickly presses the 5 button, the Cabin won't reverse its travel direction. We give the person who originally requested a down Cabin the chance to select a lower floor. P2 is the only option in our case. Once the Door closes, however, the Transfer will clear and prompt the Shaft to choose any floor high or low.



▼ Xfer13: Hold_for_call HOLDING FOR
CLEARED CALL



Door S5

LOCKED

▶ Door1: Unlock Unlocking

The Door open duration is loaded with the Bank's specified Passenger load time and the Door retry counter is reset to zero.



▼ Door13: Open_on_arrival OPENING

We decrement the max Retries counter and command the equipment to open.

SIO::Open_door (Shaft:"S5")

We also notify the UI that the door is now opening.

UI::Door_opening (Shaft_ID:"S5")

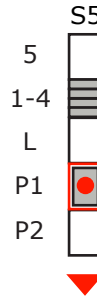
Signal I/O

The SIO domain opens the Door and also triggers a Door3: Opened event when the physical Door has fully opened.



▶ Door3: Door_opened OPEN DELAY

Now the Door is fully opened and we arrange for a signal to be sent to us at the duration loaded into the Open_time attribute earlier. We also notify the UI of the Door's progress.



UI::Door_opened (Shaft_ID:"S5");



▶ Door9: Time_to_close HOLDING OPEN

Even though our normal open delay has expired, it is possible that a passenger is pressing on the open button. The Held attribute will be set to true if this is the case. We want to hold the Door open until the button is released. In our thread, Held is false, so we generate the release signal to self and continue.

▼ Door11: Passenger_release CLOSING DOOR

Now we tell the physical Door to close and update the UI.

UI::Door_closing (Shaft_ID:"S5")

SIO::Close_door (Shaft:"S5")

Signal I/O

The Door is commanded to close using the appropriate physical output.

At some point later, we detect either a door obstructed or a door fully closed signal. Assuming the latter, we trigger the corresponding signal in the Elevator Application domain.

G



Door S5

CLOSING DOOR

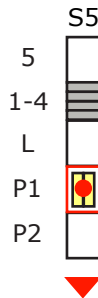
Door4: Door_closed CLOSED

When the Door closes, it checks to see if there is an active Transfer on our Cabin. If so, the Transfer is either A) unusually slow in deleting itself or B) waiting for the Door to close the first time after arrival. Either way, we notify the Transfer that the Door has closed. Since we have just serviced a down call, case B applies.

Xfer12: Door_closed ● ▼

Xfer S5-P1

In case A), the signal would just be ignored, as you can verify by looking at the Transfer state table.



The Lock_requested has not been set since there are no pending stops or calls. Even though Shaft Level S5-P1 has not yet migrated to Non Requested, it has a false Stop_requested value and no up/down link to any Bank Levels. So the Door is closed, but unlocked and the Door's participation in this thread is now complete.

F

● Xfer S5-P1

HOLDING FOR
CLEARED CALL

Xfer12: Door_closed Deleting

Now that the Door is closed, we prompt the Shaft to check both directions for possible destinations. Also, we delete ourself.



Shaft7: Transfer_completed.....>



Now that the Transfer is gone, the Shaft Level can migrate back to Non Requested status.

SLEV6: Xfer_deleted>

S5-P1





SLEV S5-P1

PENDING



SLEV6: Xfer_deleted

.....

Checking for
Unserviced Call

There is no pending call remaining
at this Shaft Level so we migrate
back to Non Requested.



SLEV4: All_requests_cleared

.....

NOT REQUESTED



Shaft S5

WAITING FOR STOP
OR CALL REQUEST



Shaft7: Transfer_completed

.....

Searching for best
destination

The Shaft scans in search of another requested Shaft Level. None
is found.



Shaft2: No_dest

.....

WAITING FOR STOP
OR CALL REQUEST

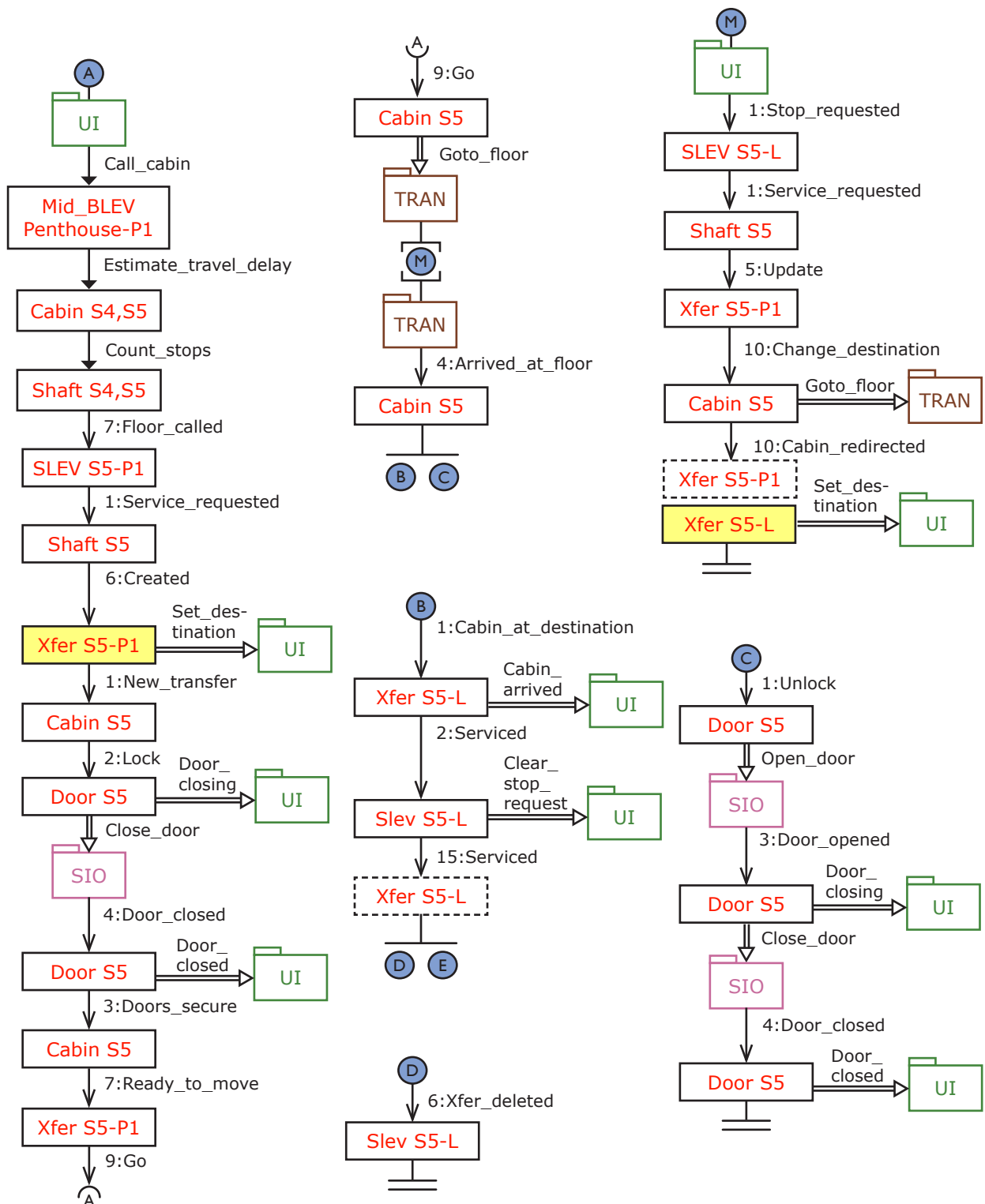


The Shaft returns to the idle state.

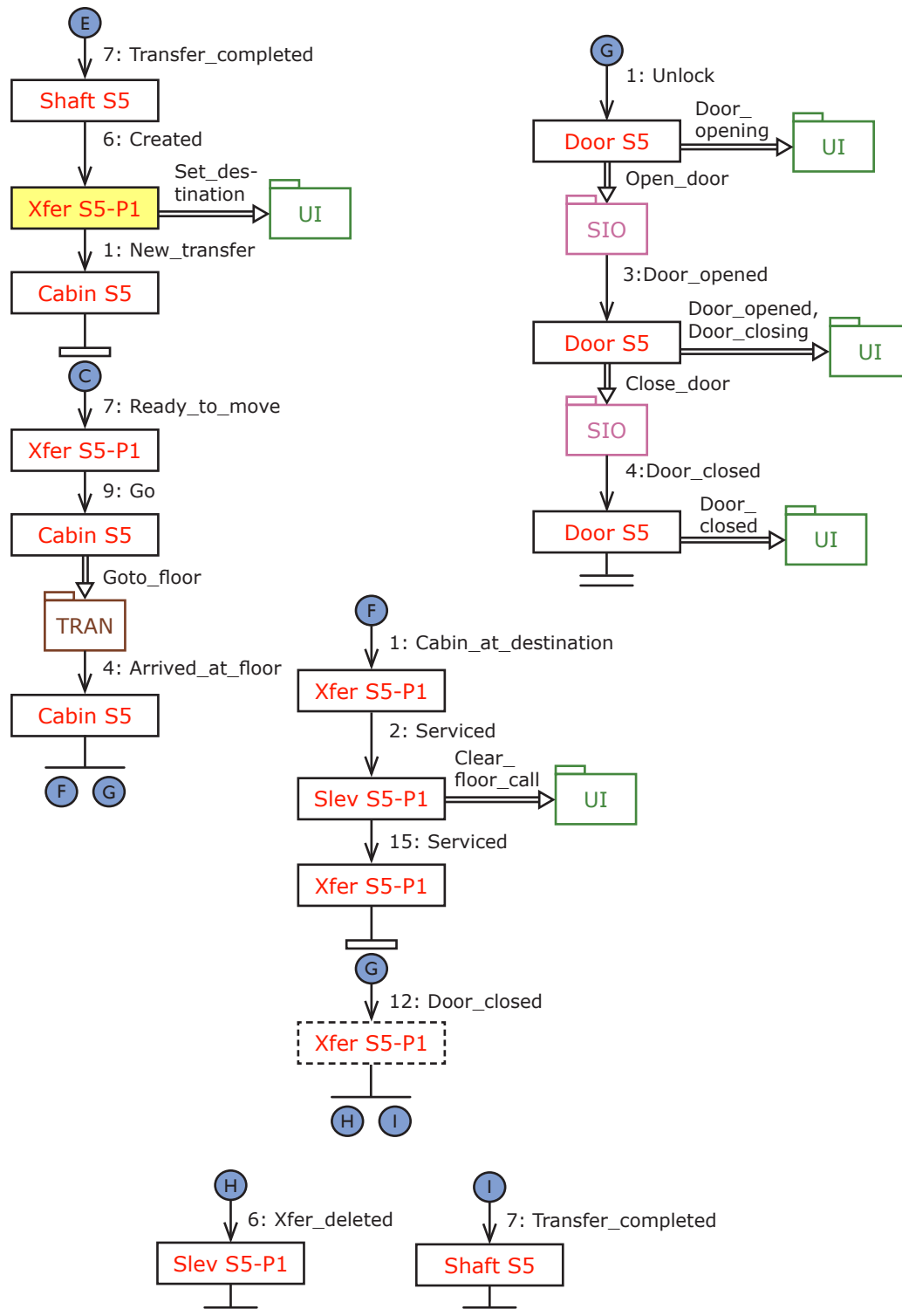
End of the F1 Thread

All strands have terminated, so this thread is complete. Here's a synopsis of what happened. A passenger at the P1 Floor pressed the down button at the Penthouse Bank. The S5 Shaft was selected to service this call. In transit to P1, a passenger inside Cabin S5 pressed the L button which triggered a stop request. Since this stop was along the way and the Cabin had not yet passed it, the P1 Transfer was converted to an L Transfer. The S5 Cabin visited the L Floor while the P1 destination was reselected. The S5 Cabin then moved to the L Floor, opened and closed its Door. Done!

Thread Chart F1: Floor Call Request



Thread Summary - Floor Call Request (continued)



Notes on the F1 Thread

1



Using a function to locate the nearest Bank Level probably seems silly. It's just the Bank Level containing the current Shaft Level, right? In this case, yes, but not always. If the Cabin is moving past the 2nd floor, de-referencing the S4-2 Shaft Level will yield an empty handle! There are no Bank Levels associated with floors 1-4 in the Penthouse Bank. So, in the Penthouse Bank, the Bank Level nearest to floor 2 is the L (lobby) level.

2



The Door thread is running independently. Here we assume the most extreme timing case where the threads C and E complete before the Door has a chance to advance out of the LOCKED state. What happens if the Lock_requested attribute is set to true and the Lock signal is sent while we are still sitting in the LOCKED state?

At first glance, it looks like trouble. You might assume that the Lock signal would be ignored and that when the Unlock signal is received, we advance to the Unlocking state and reset the Lock_requested to false! But that won't happen.

Take a look at the Door state table. You will see that the Lock signal is marked as CH - Can't happen! As it should be. But won't we crash?

No, and here's why. The Executable UML synchronization rules state that two signals generated by the same sender will be processed in the sending order. Cabin S5 sends Unlock first, then Lock. So the Door must process the Unlock signal prior to the Lock signal. Consequently, it is impossible for the Door to see a Lock signal while it is in the LOCK state. The Door will always see the Unlock signal first, which will propel it into the Unlocking state. Since the Unlocking state is transitory (exit on self-directed signal), the Lock signal won't be seen here either. The self-directed signal always fires in the Unlocking activity this self-directed signals have priority over any externally arriving signals. It is only after entry to the next non-transitory state, OPENING, that it is possible to experience a Lock event.

Bibliography

Bibliography

Click on the book title to purchase it online. Click on the primary author name at left to return to the first reference in the text.

- [\[Mellor I\]](#) Executable UML, A Foundation for Model-Driven Architecture, ISBN 0201748045, Steve Mellor and Marc Balcer, 2002, Addison-Wesley
- [\[Carter\]](#) Model Driven Architecture with Executable UML, ISBN 0521537711, Colin Carter, Chris Raistrick, et al, 2004, Cambridge University Press
- [\[UML\]](#) UML 2.0 Specifications, Object Management Group (OMG)
www.omg.org